

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1201	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	K. MRUDHULA	Reg.No.:	192524335
Department:	B. Tech AI&DS	Date:	24/08/26

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

Answer::;

Problem Understanding

The smartphone experiences lag while switching between applications because the processor must frequently access application data and instructions from memory. If frequently used data is not available in the faster cache levels, the CPU must access DRAM, which has significantly higher latency than cache memory. Efficient use of the memory hierarchy is therefore required to reduce access latency and improve application-switching performance.

Analysis of Existing Memory Hierarchy

Memory	Relative Speed	Capacity	Main Role
L1 Cache	Very high	Very small	Frequently accessed instructions/data
L2 Cache	High	Small/medium	Backup for L1

Memory	Relative Speed	Capacity	Main Role
L3 Cache	Moderate	Larger	Shared CPU-level cache
DRAM	Lower	Large	Main memory for applications

L1 cache should store the most frequently accessed instructions and data because it provides the lowest latency.

L2 cache provides additional storage when the required information is not available in L1.

L3 cache acts as a larger shared cache and reduces the number of accesses to DRAM.

DRAM provides large-capacity storage for running applications but has considerably higher access latency than cache memory.

Proposed Hierarchical Redesign

The smartphone can use the following hierarchy:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM → Storage

1. Increase the effectiveness of L1 cache by optimizing cache size and associativity.
2. Use a moderately larger L2 cache to store application data displaced from L1.
3. Increase L3 cache capacity to reduce expensive DRAM accesses during multitasking.
4. Use high-bandwidth, low-power mobile DRAM such as LPDDR.
5. Apply cache-aware scheduling so frequently used application data remains closer to the processor.
6. Use prefetching for predictable memory access patterns.

Tools and Performance Evaluation

Tools such as Linux perf, Cachegrind, or CPU/cache simulators can be used to measure cache hit rate, cache misses, memory latency and CPU utilization.

Important measurements include:

- L1/L2/L3 cache hit rate
- Cache miss rate
- DRAM access frequency
- Memory bandwidth

- Application-switching latency

Expected Result

The redesigned hierarchy reduces the number of DRAM accesses and keeps frequently used application data closer to the CPU. This decreases average memory access time, improves multitasking responsiveness and increases effective throughput.

Conclusion:

A balanced L1–L2–L3 cache + high-bandwidth DRAM hierarchy provides the best solution because cache minimizes latency while DRAM provides the required capacity.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

Answer:

Problem Understanding

The cloud provider experiences high database-query latency. Database workloads repeatedly access frequently used indexes, records and query instructions. If this frequently accessed information is found in cache, the CPU can access it quickly. However, if the working set exceeds available physical memory, virtual memory paging may cause page faults and significantly increase latency.

Cache Hierarchy vs Virtual Memory Paging

Feature	Cache Hierarchy	Virtual Memory Paging
Purpose	Reduce CPU memory-access latency	Extend usable memory
Main mechanism	Cache lines	Pages
Speed	Very high	Much slower during page faults
Managed by	Hardware/CPU	OS + MMU
Typical issue	Cache miss	Page fault

Feature	Cache Hierarchy	Virtual Memory Paging
Best for	Frequently accessed data	Large memory requirements

The cache hierarchy should therefore be optimized before relying heavily on paging.

Engineering Solution

The recommended architecture is:

CPU → L1 → L2 → L3 → DRAM → SSD/Storage

1. Keep frequently accessed database indexes and query data in cache where possible.
2. Increase or optimize L3 cache for shared database workloads.
3. Provide sufficient DRAM to accommodate the database working set.
4. Reduce unnecessary paging by monitoring page-fault rates.
5. Use virtual memory efficiently through proper page allocation.
6. Use fast SSD storage as backing storage when paging cannot be avoided.
7. Apply database buffer caching to keep frequently accessed records in RAM.

Appropriate Memory Technologies

SRAM/cache:

Best for very low-latency processor-level access.

DRAM:

Best for large-capacity working memory and high-bandwidth database operations.

NAND Flash/SSD:

Provides persistent storage but has substantially higher latency than DRAM.

Tools

The cloud provider can use:

- perf for CPU/cache performance
- Linux vmstat for paging activity
- iostat for storage performance
- Database monitoring tools for query latency
- Memory profiling tools for working-set analysis

Analysis

Suppose a database query frequently accesses the same index. If the index remains in cache, the CPU avoids repeated DRAM accesses. If physical memory becomes insufficient, pages may be moved between DRAM and storage. Frequent page faults then increase query latency dramatically.

Therefore, adding sufficient DRAM and improving cache locality is preferable to simply increasing paging activity.

Conclusion

The recommended solution is to maximize cache hit rates, provide sufficient DRAM for the active database working set, and minimize page faults. Virtual memory should act as a capacity-extension mechanism rather than the primary solution for performance.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Answer:

Problem Understanding

An autonomous vehicle continuously receives data from cameras, LiDAR, radar and other sensors. This data must be processed in real time. Any significant memory delay can increase processing latency and potentially affect decision-making. Therefore, the memory system must provide very low latency and high bandwidth.

Comparison of SRAM, DRAM and MRAM

Technology	Latency	Volatility	Capacity	Suitable Application
SRAM	Very low	Volatile	Low	CPU cache
DRAM	Low/medium	Volatile	High	Main memory
MRAM	Low	Non-volatile	Medium	Fast persistent/embedded memory

SRAM is ideal for processor caches because of its extremely low access latency.

DRAM provides the large capacity required for sensor data and AI workloads.

MRAM provides non-volatile storage with fast access and can retain data without continuous power.

Proposed Memory Hierarchy

The recommended structure is:

CPU/AI Accelerator → SRAM L1/L2 Cache → Larger SRAM/L3 → DRAM → MRAM

- **L1 SRAM:** Most frequently accessed sensor-processing instructions and data.
- **L2/L3 SRAM:** Larger working datasets shared by processing cores.
- **DRAM:** Large sensor buffers, AI model data and intermediate processing results.
- **MRAM:** Fast non-volatile storage for critical configuration, frequently required persistent data and recovery information.

Design Improvements

1. Use multi-level SRAM caches close to processing cores.
2. Use high-bandwidth DRAM for large sensor-data buffers.
3. Apply DMA to transfer sensor data efficiently without excessive CPU involvement.
4. Use memory prefetching for predictable sensor streams.
5. Use MRAM for selected non-volatile workloads.
6. Ensure memory controllers provide sufficient bandwidth for simultaneous sensor streams.

Tools and Validation

Tools such as gem5, Cachegrind and hardware performance counters can simulate or measure:

- Cache hit rate
- Memory bandwidth
- Access latency
- CPU utilization
- Sensor-processing throughput

Analysis

For real-time autonomous systems, latency and bandwidth are both important. SRAM minimizes latency, while DRAM provides capacity and bandwidth. MRAM adds non-volatility without depending completely on slower storage technologies.

Conclusion

A SRAM → DRAM → MRAM hierarchy provides a balanced solution. SRAM handles latency-sensitive processing, DRAM handles large high-bandwidth datasets, and MRAM provides fast non-volatile storage.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

Answer:

Problem Understanding

An edge AI device must load machine-learning models quickly while consuming limited power. Model files are generally stored in non-volatile memory, while active inference requires fast working memory. Therefore, NAND Flash, DRAM and RRAM must be appropriately organized.

Comparison

Technology	Speed	Power	Volatility	Main Use
NAND Flash	Moderate/slow compared w RAM	Low	Non-volatile	Model storage
DRAM	High	Higher than Flash	Volatile	Runtime model/data
RRAM	High potential	Low	Non-volatile	Fast model storage/working memory

Proposed Hierarchical Structure

AI Processor → SRAM/Cache → DRAM → RRAM → NAND Flash

Although the main technologies specified are NAND Flash, DRAM and RRAM, a small processor cache can be placed above them for extremely latency-sensitive operations.

Role of Each Memory

NAND Flash

- Stores complete AI models.
- Provides high storage density.
- Retains data when power is removed.
- Suitable for large model storage.

DRAM

- Stores the active model during inference.
- Holds intermediate tensors and runtime data.
- Provides high bandwidth.
- Provides faster access than NAND Flash.

RRAM

- Provides non-volatile storage with fast access.
- Can reduce dependence on slower NAND Flash.
- Can be used for frequently loaded model parameters.
- Helps reduce data movement and potentially power consumption.

Optimization Strategy

1. Store the complete model in NAND Flash.
2. Move frequently used model components to RRAM.
3. Load active inference data into DRAM.
4. Keep frequently accessed data close to the AI processor.
5. Use model compression and quantization to reduce memory requirements.
6. Reduce unnecessary transfers between NAND Flash and DRAM.

Tools

Performance can be evaluated using:

- AI inference benchmarks
- Memory-bandwidth monitoring
- Power-monitoring tools
- Cache/memory simulators

- Hardware performance counters

Important metrics include:

Model loading time, inference latency, memory bandwidth, power consumption and throughput.

Analysis

NAND Flash provides capacity but is slower than RAM. DRAM provides high-speed runtime access but is volatile. RRAM can bridge part of this gap by providing fast non-volatile storage.

Conclusion

The optimal design is a DRAM + RRAM + NAND Flash hierarchy, where NAND Flash provides high-capacity storage, RRAM provides fast non-volatile access, and DRAM supports high-speed inference execution.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Answer:

Problem Understanding

The gaming console experiences frame drops while loading large scenes. Modern games continuously transfer textures, geometry, shaders and other assets into memory. If memory bandwidth is insufficient or data cannot be supplied to the processor quickly enough, the rendering pipeline may stall, producing frame drops.

L3 Cache vs DRAM

Feature	L3 Cache	DRAM
Latency	Very low	Higher
Capacity	Smaller	Much larger
Bandwidth	High for CPU-local access	High overall memory bandwidth
Purpose	Frequently reused data	Large game datasets

L3 cache is useful for frequently reused CPU data, but its limited capacity prevents it from storing complete large game scenes.

DRAM provides the capacity and bandwidth needed for large textures, game assets and rendering data.

Proposed Memory Hierarchy

CPU/GPU → L1 → L2 → L3 → High-Bandwidth DRAM → Storage

Enhancements

1. Increase effective L3 cache capacity to retain frequently reused game data.
2. Use high-bandwidth DRAM to supply large scene assets quickly.
3. Optimize memory-controller scheduling.
4. Use prefetching to load predictable scene data before it is required.
5. Keep frequently reused CPU/game-engine data in cache.
6. Use fast persistent storage for rapid asset loading into DRAM.
7. Optimize data layout to improve spatial and temporal locality.

Tools and Performance Testing

The console can use:

- GPU/CPU performance counters
- Frame-time profilers
- Memory-bandwidth monitors
- Cache-miss analysis
- Game-engine profiling tools

Important measurements include:

- Average frame time
- 1% low FPS
- L3 cache hit rate
- DRAM bandwidth utilization
- Cache-miss rate
- Scene-loading latency

Analysis

Increasing only L3 cache will not solve the problem if the primary limitation is the transfer of large scene assets. L3 is valuable for reducing CPU memory latency, but high-bandwidth DRAM is essential for large-volume data transfer.

Therefore, the best solution is to optimize both cache locality and DRAM bandwidth.

Conclusion

A high-performance gaming console should use a large and efficient L3 cache combined with high-bandwidth DRAM and fast storage. L3 reduces latency for frequently reused data, while DRAM provides the bandwidth and capacity required for large gaming scenes. This reduces rendering stalls and helps maintain stable frame rates.

Code of Conduct:

*I **MRUDHULA K (192524335)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MRUDHULA

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis